

Auto-generated PDF — built from `ANALYTICS.md` (the editable source of truth) and `src/lib/analytics.js` (the runtime). Regenerated by CI on every change to either file.

Built 2026-05-17 01:54 UTC · branch `claude/busy-pare-a438c2` · commit `7b8e4f7`

Analytics — Tracking Plan & GA4 Setup

This document describes what events PSPO-I Trainer fires, what they mean, and how to configure the GA4 property to surface the most useful reports.

The implementation lives in `src/lib/analytics.js`. Events are routed:

```
App code → window.dataLayer.push({event, ...}) → GTM (GTM-THK4XJZW) → GA4 (G-WB97T5NR8S)
```

Every event is consent-gated through CookieYes (the `analytics` category). We do **not** rely on CookieYes's `type="text/plain" + data-cookieyes` attribute mechanism for script gating — in practice it silently fails on SPAs even after the user accepts. Instead, the loader in `index.html` reads CookieYes's consent cookie directly and only inserts the GTM script tag once `analytics:yes` is present. It listens for CookieYes's `cookieyes_consent_update` / `cookieyes_consent_set` events and also polls (500 ms × 120 = 60 s) as a safety net.

1. Tracking plan

Events

All events are fired client-side via `gtag('event', name, params)`.

Event	Fires when	Key parameters
page_view	View changes (SPA: title → home → lesson → quiz → results → ...). gtag's auto page_view is disabled ; this is the sole source.	page_path (/quiz/scrum_theory), page_title , page_location
theme_switch	User toggles between classic and arcade	from , to
concept_view	User opens a concept's lesson page	concept_id , concept_label
quiz_start	Any quiz starts	mode ∈ { concept , mixed , mock , review }, concept_id? , concept_label? , question_count
quiz_complete	Any quiz finalizes	mode , concept_id? , concept_label? , score_pct , correct_count , wrong_count , total_questions , time_used_sec? , verdict , value (=score_pct)
quiz_pass	Quiz finalizes with score_pct >= 85 (PSPO I pass bar). Fires in addition to quiz_complete .	same as quiz_complete
mock_exam_complete	A mock-exam (mode= mock) finalizes. Fires in addition to quiz_complete .	same as quiz_complete
quiz_abandon	User exits a quiz before finishing — via the back button, the header nav, or any other navigation away from the quiz view. Mutually exclusive with quiz_complete for any given quiz session.	mode , concept_id? , concept_label? , question_count , answered_count
concept_mastered	A concept's mastery level transitions to mastered . Detected by diffing the mastered-set on every progress mutation.	concept_id , concept_label
achievement_unlocked	First time an achievement unlocks. Idempotent across sessions via localStorage (pspo_flawless_event_fired and pspo_mock_complete_unlocked). Currently: - flawless_victory — finished any quiz with correct === total - mock_complete — all required concepts mastered	achievement_id , achievement_name

User properties

Set on app boot and re-set whenever they change:

Property	Type	Source
theme	string (classic arcade)	current theme
concepts_mastered	integer (0–10)	count of concepts at mastery level
total_progress_pct	integer	overallProgress(progress).total

Dedupe contract — how each event is guarded against duplicate firing

Event	Dedupe mechanism
page_view	React <code>useEffect([view, activeConcept, loaded])</code> . Fires when the dependency tuple changes — no other code path emits page_view.
set_user_properties	Single consolidated <code>useEffect([theme, progress, loaded])</code> with a <code>useRef</code> snapshot of the last pushed payload. Fires only when at least one of <code>{theme, concepts_mastered, total_progress_pct}</code> actually changes.
theme_switch	Inline in <code>switchTheme</code> . The accompanying user-property update is handled by the <code>set_user_properties</code> effect — <code>trackThemeSwitch</code> does not call <code>setUserProperties</code> to avoid double-pushing.
concept_view	Inline in <code>pickConcept</code> . One per click.
quiz_start	Inline in <code>startQuizForConcept</code> / <code>startQuickQuiz</code> / <code>startMockExam</code> / <code>startReview</code> . One per click.
quiz_complete (and derived quiz_pass , mock_exam_complete)	Single fire from each shell: • Arcade: <code>ArcadeShell.finishQuiz</code> (called once by <code>QuizScreen.finalize</code>) • Classic: <code>QuizView</code> <code>useEffect([finished])</code> with a <code>reportedFinishRef</code> so a re-render with <code>finished===true</code> doesn't re-fire.
concept_mastered	<code>useEffect([progress, loaded])</code> with <code>prevMasteredRef</code> . Diffs <code>mastered now</code> vs <code>mastered last run</code> ; fires only for newly-mastered concept ids. The initial <code>null</code> ref guarantees no phantom events for returning users with already-mastered concepts.
achievement_unlocked	Stateful idempotency: • <code>flawless_victory</code> — guarded by <code>localStorage.pspo_flawless_event_fired</code> so subsequent flawless quizzes don't re-fire. • <code>mock_complete</code> — guarded by <code>localStorage.pspo_mock_complete_unlocked</code> AND a same-session ref.
quiz_abandon	Called from <code>maybeTrackQuizAbandon()</code> , invoked by both <code>exitQuiz</code> and the header nav handler <code>navigateTo</code> . Only fires when <code>view === 'quiz'</code> && <code>!qsess.finished</code> . Mutually exclusive with <code>quiz_complete</code> for any quiz session.

When adding a new event, write down its dedupe story here. If you can't, the code probably duplicates somewhere.

Verdicts (the `verdict` parameter)

Computed by `verdictFor(scorePct)` in `src/lib/quiz.js` :

score_pct	verdict
≥ 95	perfect
≥ 85	pass
≥ 70	close
< 70	fail

2. Setting up GTM and GA4 — step-by-step

Do this once. Roughly 30–45 minutes the first time. Open tagmanager.google.com and select container `GTM–THK4XJZW` , and a second tab for analytics.google.com on property `G–WB97T5NR8S` .

2.1 Configure the base Google tag (in GTM)

This is the single GA4 base tag that everything else reuses.

1. GTM → **Tags** → **New**
2. Tag Configuration → **Google Tag** (not "GA4 Event" — the higher-level Google Tag covers both)
3. Tag ID: `G–WB97T5NR8S`
4. **Configuration settings** → add row:
 - Name: `send_page_view`
 - Value: `false` *(Why: the SPA fires its own `page_view` for every screen change. If we leave the auto `page_view` on, the first screen would be double-counted.)*
5. **Shared event settings** → (leave empty — we'll set per-event params in each event tag)
6. Triggering → **Initialization - All Pages** (fires once when the page bootstraps, before any custom events). If you don't see it, choose **Consent Initialization - All Pages**.
7. Name the tag: `GA4 – Google tag (base config)` . Save.

2.2 Create Data Layer Variables (in GTM)

Variables let GTM read parameters off each `dataLayer.push` . Create one per parameter the app sends — name them with the `dlv_` prefix for clarity.

GTM → **Variables** → **User-Defined Variables** → **New** for each row below. Each one is a **Data Layer Variable**, **Data Layer Variable Name** = the column on the right, **Data Layer Version** = 2.

Variable name	Reads dataLayer key
dlv_mode	mode
dlv_concept_id	concept_id
dlv_concept_label	concept_label
dlv_question_count	question_count
dlv_score_pct	score_pct
dlv_correct_count	correct_count
dlv_wrong_count	wrong_count
dlv_total_questions	total_questions
dlv_time_used_sec	time_used_sec
dlv_verdict	verdict
dlv_achievement_id	achievement_id
dlv_achievement_name	achievement_name
dlv_from	from
dlv_to	to
dlv_page_path	page_path
dlv_page_title	page_title
dlv_page_location	page_location
dlv_theme	theme
dlv_concepts_mastered	concepts_mastered
dlv_total_progress_pct	total_progress_pct
dlv_answered_count	answered_count

2.3 Create Triggers (in GTM)

One **Custom Event** trigger per event the app emits. GTM → **Triggers** → **New** for each row.

For each: Trigger Configuration → **Custom Event**, **Event name** = exact string in the right column, **This trigger fires on** = All Custom Events.

Trigger name	Event name (exact match)
CE — page_view	page_view
CE — theme_switch	theme_switch
CE — concept_view	concept_view
CE — quiz_start	quiz_start
CE — quiz_complete	quiz_complete
CE — quiz_pass	quiz_pass
CE — mock_exam_complete	mock_exam_complete
CE — concept_mastered	concept_mastered
CE — achievement_unlocked	achievement_unlocked
CE — quiz_abandon	quiz_abandon
CE — set_user_properties	set_user_properties

2.4 Create GA4 Event tags (in GTM)

One **GA4 Event** tag per trigger above. Each tag forwards the event + its parameters to GA4. GTM → **Tags** → **New** for each row in the table below.

For each tag:

- Tag Configuration → **Google Analytics: GA4 Event**
- **Measurement ID:** G-WB97T5NR8S
- **Event Name:** the GA4 event name (right column)
- **Event Parameters:** add a row per parameter listed for that tag, Name = parameter name, Value = the matching `{{dlv_*}}` variable
- **Triggering:** the matching `CE — *` trigger from §2.3

Tag name	GA4 Event Name	Event Parameters to map
GA4 – page_view	page_view	page_path = {{dlv_page_path}}, page_title = {{dlv_page_title}}, page_location = {{dlv_page_location}}
GA4 – theme_switch	theme_switch	from = {{dlv_from}}, to = {{dlv_to}}
GA4 – concept_view	concept_view	concept_id = {{dlv_concept_id}}, concept_label = {{dlv_concept_label}}
GA4 – quiz_start	quiz_start	mode, concept_id, concept_label, question_count (map each to its dlv_*)
GA4 – quiz_complete	quiz_complete	mode, concept_id, concept_label, score_pct, correct_count, wrong_count, total_questions, time_used_sec, verdict, plus value = {{dlv_score_pct}}
GA4 – quiz_pass	quiz_pass	same as quiz_complete
GA4 – mock_exam_complete	mock_exam_complete	same as quiz_complete
GA4 – concept_mastered	concept_mastered	concept_id = {{dlv_concept_id}}, concept_label = {{dlv_concept_label}}
GA4 – achievement_unlocked	achievement_unlocked	achievement_id = {{dlv_achievement_id}}, achievement_name = {{dlv_achievement_name}}
GA4 – quiz_abandon	quiz_abandon	mode = {{dlv_mode}}, concept_id = {{dlv_concept_id}}, concept_label = {{dlv_concept_label}}, question_count = {{dlv_question_count}}, answered_count = {{dlv_answered_count}}

2.5 Map user properties (in GTM)

User properties are set via a dedicated `set_user_properties` event the app fires whenever theme/concepts_mastered/total_progress_pct change.

Create one final tag:

- GTM → Tags → New
- Tag Configuration → Google Analytics: GA4 Event
- Measurement ID: G-WB97T5NR8S
- Event Name: `set_user_properties` (GA4 ignores the event but accepts the user property payload)
- Expand **User properties** → add three rows:
 - `theme = {{dlv_theme}}`
 - `concepts_mastered = {{dlv_concepts_mastered}}`
 - `total_progress_pct = {{dlv_total_progress_pct}}`
- Triggering: CE – `set_user_properties`
- Name it: **GA4 – set user properties** . Save.

2.6 GA4 console — register custom definitions

Custom dimensions/metrics let your event params show up in GA4 reports. Without this step the data is collected but invisible.

GA4 → Admin → Custom definitions → Create custom dimensions (scope: **Event**):

Dimension name	Event parameter
Quiz mode	mode
Concept	concept_label
Concept ID	concept_id
Verdict	verdict
Achievement	achievement_name

GA4 → Admin → Custom definitions → Create custom metrics (scope: **Event**, Unit: **Standard**):

Metric name	Event parameter
Score %	score_pct
Correct count	correct_count
Wrong count	wrong_count
Time used (sec)	time_used_sec
Total questions	total_questions

GA4 → Admin → Custom definitions → Create custom dimensions (scope: **User**):

Dimension name	User property
Theme (user)	theme
Concepts mastered	concepts_mastered
Total progress %	total_progress_pct

2.7 GA4 console — mark key events (conversions)

GA4 → Admin → Events. Find each of these in the list and toggle **Mark as key event**:

- quiz_pass
- mock_exam_complete
- concept_mastered
- achievement_unlocked

If the event hasn't fired yet (no traffic), use **Create event** → "Mark as key event" with the exact name and a condition like `event_name equals quiz_pass` so it's pre-registered.

2.8 GA4 console — retention + privacy

- Admin → Data settings → Data retention → 14 months (max for free GA4)
- Reset on new activity: On

- IP anonymization is automatic in GA4; nothing to toggle.

2.9 Test in GTM Preview mode (no traffic needed)

This is the key part of "before any traffic":

1. GTM → top-right **Preview** button
2. A new tab opens — paste your dev or staging URL (`http://localhost:5174`)
3. The real site opens in another tab; the **Tag Assistant** panel shows it's connected
4. In the site, **accept** analytics in the CookieYes banner. GTM should immediately appear in the **Tags Fired** list (alongside **GA4 — Google tag (base config)**)
5. Navigate: title → home → click a concept → click Start Quiz → answer questions. After each action, check the Tag Assistant left panel — each user action should show one or more tags firing
6. Also open GA4 → **Reports** → **Real-time** → **DebugView**. Your events should appear there within ~10 seconds. Click an event to see its parameters
7. If something's missing, check:
 - The trigger's "Event name" exactly matches the string the app pushes (case-sensitive)
 - The DLV's "Data Layer Variable Name" exactly matches the key in the push
 - The tag is referenced by the trigger

2.10 Publish the GTM container

Once everything fires correctly in Preview:

1. GTM → top-right **Submit**
2. Version name: e.g. `v1 — analytics rollout`
3. Version description: paste a link to this `ANALYTICS.md`
4. **Publish**

Tags now fire in production. Real users immediately start populating GA4 (subject to consent).

3. Recommended explorations

Build these once in GA4 → Explore. Each is a free-form exploration unless noted.

3.1 "Quiz mode mix"

Question: *Which quiz modes do users actually start?*

- Rows: `Quiz mode` (custom dim)
- Values: Event count for `quiz_start` , Event count for `quiz_complete`
- Add a calculated column: completion rate = `quiz_complete / quiz_start`

3.2 "Pass rate by quiz mode"

Question: *How often do users pass each mode?*

- Filter: Event name = `quiz_complete`
- Rows: `Quiz mode` , `Verdict`
- Values: Event count
- Pivot `Verdict` horizontally for an at-a-glance heatmap

3.3 "Hardest concepts"

Question: *Which concepts have the lowest average score on concept-mode quizzes?*

- Filter: Event name = `quiz_complete` AND `Quiz mode` = `concept`
- Rows: `Concept`
- Values: `avg( score_pct )` , `sum( wrong_count )` , Event count
- Sort by `avg( score_pct )` ascending

3.4 "Mock exam funnel"

Question: *How many starts convert to completes and to passes?*

- Funnel exploration
- Steps:
 1. `quiz_start` with `mode = mock`
 2. `mock_exam_complete`
 3. `quiz_pass` with `mode = mock`

3.5 "Theme preference"

Question: *Which theme do power users prefer?*

- Rows: `Theme (user)`
- Values: Active users, Average `concepts_mastered` , Event count of `quiz_complete`

3.6 "Average time-to-mastery"

Question: *How long between first quiz start and concept mastery?*

- Event count of `concept_mastered` over time
- Cross-reference with first-seen date from User Acquisition

3.7 "Engagement depth"

Question: *Do returning visitors complete more quizzes than new ones?*

- Rows: New / Returning (built-in)

- Values: avg quizzes/user, avg score, Active users

4. Privacy & consent

- The GTM loader script in `index.html` carries `type="text/plain"` + `data-cookieyes="cookieyes-analytics"`. It sits dormant in the DOM until CookieYes flips the type to `text/javascript` after consent.
- Pre-consent, GTM never loads, so no tags fire and no requests leave the browser. The app still pushes events to `window.dataLayer` but they pile up locally and are discarded on reload.
- Post-consent, GTM boots and replays the queued `dataLayer` entries through its tag pipeline — so events that happened *during* the consent decision aren't lost.
- No PII is ever sent. Concept labels and question counts are the most identifying things in our payloads; no user-input text, no emails, no IPs beyond GA4's automatic (and anonymized) capture.
- Page paths are synthetic SPA routes (`/title` , `/home` , `/lesson/scrum_theory`), not URLs the user typed.

5. Troubleshooting — "tracking isn't working"

There are exactly four places this pipeline can break. Run through them in order; the symptoms below tell you which step you're stuck on.

5.1 `dataLayer` pushes happen?

Open DevTools → Console, run `window.dataLayer` and look for entries with `event` keys like `page_view` , `set_user_properties` , `quiz_start` . If you see them, the **app layer is fine**. If `window.dataLayer` is undefined or empty after navigating, the inline init in `index.html` is not running — check for a syntax error in your edits or a CSP that blocked the inline script.

5.2 GTM loaded? (the most common gotcha)

Run `Object.keys(window).filter(k => /gtm|google_tag/.test(k))` in the console. You should see `google_tag_data` and `google_tag_manager` . If you see *nothing*, GTM hasn't loaded.

Why this usually happens: CookieYes only renders its consent banner on the *exact* production domain registered in its dashboard. On any other origin — localhost, every Vercel preview deploy (`p5-p0-git-<branch>-<user>.vercel.app`), staging — the CookieYes script loads but silently no-ops, which means the `type="text/plain"` GTM snippet never gets activated.

Fix in dev and on Vercel previews (automatic): The non-production bootstrap block in `index.html` keys off Vercel's build-time `VERCEL_ENV` value (forwarded into the bundle as `VITE_VERCEL_ENV` by `vite.config.js`):

VERCEL_ENV	Behavior
production	Skip the bootstrap. Real users go through CookieYes as designed.
preview	Auto-load GTM. Preview deploys can be tested with GA4 DebugView.
development	Auto-load GTM (this is the default Vercel injects on <code>vercel dev</code> and what we fall back to locally).

When testing on a preview deploy, start a **GTM Preview session** from `tagmanager.google.com` so the events are tagged `debug_mode=1` and route to GA4 DebugView only — not your production reports.

Fix in production: In the CookieYes dashboard (app.cookieyes.com) → Site settings → **Site URL**, register your production deploy URL exactly (including `https://` and no trailing slash). For a Vercel project without a custom domain that's the default `<project>.vercel.app`; with a custom domain, it's that domain. If neither matches what the user lands on, the banner won't render and GTM won't activate.

5.3 GTM tags configured?

Even with GTM loaded, you need the tags inside the GTM container to forward events to GA4. If you see GTM in the dataLayer but no events in GA4 DebugView, it's almost certainly this.

Open `tagmanager.google.com` → container `GTM-THK4XJZW` → **Workspace Changes / Versions**. If the only thing there is the default empty workspace, you haven't done the §2 setup yet. Follow §2.1 through §2.10 to wire the tags, then **Submit / Publish**.

Quickest verification: in GTM, click **Preview** → enter your URL → click around in your app. Each user action should make tags fire in the Tag Assistant left rail. If nothing fires, the trigger event names don't match what the app pushes (case-sensitive).

5.4 GA4 receiving events?

Open GA4 → Reports → **Real-time** → **DebugView**. Events appear within ~10 seconds. If GTM Preview shows tags firing but DebugView is empty:

- The GA4 Event tag is missing its Measurement ID (should be `G-WB97T5NR8S` on every event tag)
- The browser has DNT or an ad blocker that's blocking `google-analytics.com/g/collect`
- The GA4 property's data stream is paused (Admin → Data streams)

5.5 Quick diagnostic snippet

Paste into the DevTools console to summarize the state of the pipeline in one go:

```
``js ({ dataLayerLen: window.dataLayer?.length, appEventsQueued: window.dataLayer?.filter(e => e.event && !e.event.startsWith('gtm.')).map(e => e.event), gtmLoaded: !!window.google_tag_manager, gtmGated: !!document.querySelector('script[type="text/plain"][data-cookieyes]'), cookieYesActive: !!document.querySelector('.cky-consent-bar, [id^="cky-"]'), host: location.hostname, }) ``
```

If `gtmGated: true` and `gtmLoaded: false`, you're hitting issue 5.2 (consent gate is blocking GTM). If `gtmLoaded: true` and you don't see events in GA4 DebugView, you're hitting 5.3 (no tags configured) or 5.4 (downstream).

6. Debugging

- **Did the `dataLayer` push fire?** Open DevTools → Console and inspect `window.dataLayer` . Each event is a separate array entry. Run `window.dataLayer.slice(-5)` to see the last five pushes.
- **Did GTM forward it to GA4?** Open GTM → Preview, paste your URL, accept analytics, and watch the Tag Assistant. Each tag that fires shows the event name and the parameters it sent.
- **Real-time view:** GA4 → Reports → Real-time → DebugView. To force debug mode without the GA Debugger extension, paste this in the console after accepting analytics:

```
window.dataLayer.push({ event: 'debug_mode_on', debug_mode: true });
```

(Then configure a GTM tag that maps `debug_mode` onto the GA4 Config — or install the [GA Debugger Chrome extension](#).)

- **Consent dry-run:** in DevTools → Application → Cookies, delete the `cookieyes-consent` cookie and reload — the banner reappears so you can re-test the pre-consent and post-consent paths.

7. PDF version (auto-generated)

A rendered PDF lives at `ANALYTICS.pdf` in the repo root — the same content as this file, formatted for printing or sharing with stakeholders who don't read markdown.

It is rebuilt automatically:

- **Locally:** `npm run docs:analytics` (uses `md-to-pdf`)
- **In CI:** `.github/workflows/build-analytics-pdf.yml` re-runs the script on every push to `main` that touches `ANALYTICS.md` , `src/lib/analytics.js` , or the build script itself, and commits the regenerated PDF back to `main` with `[skip ci]` . The PDF's header banner shows the build timestamp + commit SHA so the reader can verify it's current.

The markdown is the editable source of truth; the PDF is a derived artifact — never edit it by hand.

8. Future work

Out of scope for this pass, but worth considering:

- **Scroll depth on long lessons.** Fire `lesson_section_view` for each section that scrolls into view. Useful only if average reading time shows users dropping off.
- **Per-question analytics.** Currently we don't fire per-answer events — the data is too noisy and most of it lives in `localStorage` anyway. If a future need surfaces (e.g., "which questions are universally hardest"), an `question_answered` event with `concept_id` , `difficulty` , `correct` would do it.
- **Server-side measurement protocol** for offline events (e.g., if we ever email reminders).
- **Google Consent Mode v2 explicit signals.** Today we rely on CookieYes to gate the GTM loader entirely (a stronger guarantee than Consent Mode's "denied" state). If we ever need anonymous pings pre-consent (e.g. cookieless pings

to estimate top-line traffic), remove the `data-cookieyes` block on the GTM script and configure Consent Mode v2 in GTM via the **Consent** tag template — CookieYes has a native integration for this in its dashboard.

- **Container as code.** Export the GTM container JSON (Admin → Export Container) into the repo as `.gtm/container.json` so the container config is reviewable in PRs and restorable on accidental deletion.